

Алгоритмы и структуры данных

2 семестр (ITMO IS)

Обход в глубину (DFS)

DFS строит дерево обхода, заходя в каждого соседа до упора, а затем откатываясь назад.

```
void dfs(int v) {
    used[v] = true;
    for(int to: g[v])
        if(!used[to]) dfs(to);
}
```

Время: $O(V + E)$.

Обход в ширину (BFS)

BFS обходит граф слоями от стартовой вершины и поэтому в невзвешенном графе сразу даёт кратчайшее расстояние в числе рёбер.

```
queue<int> q;
q.push(s); dist[s]=0;
while (!q.empty()){
    int v = q.front();
    q.pop();
    for(int to: g[v])
        if(dist[to]==INF {
            dist[to]=dist[v] + 1;
            p[to] = v;
            q.push(to);
        }
}
```

Время: $O(V + E)$.

Топологическая сортировка

Топологическая сортировка задаёт порядок вершин в DAG, при котором каждое ориентированное ребро идёт из более ранней вершины в более позднюю.

```
vector<int> order(v);
vector<int> used(v);
void dfs(int v){
    used[v] = 1;
    for(int to: g[v]) {
        if(!used[to]) dfs(to);
    }
    order.push_back(v);
}

void dag() {
    for (int i = 0; i < v; i++) {
        if(!used[i]) {
            dfs(i);
        }
    }
    reverse(order.begin(), order.end());
}
```

Время: $O(V + E)$.

Конденсация графа. Поиск компоненты слабой связности

Сильные компоненты связности (SCC) объединяют вершины, достижимые друг из друга. Алгоритм Косарайо: первый DFS строит порядок выхода, второй DFS на транспонированном графе в обратном

порядке выхода выделяет компоненты. Слабые компоненты — SCC на неориентированной версии графа.

```
// 1. DFS на G: записать вершины в порядке выхода
void dfs1(int v){
    used[v]=1;
    for(int to: g[v]) if(!used[to]) dfs1(to);
    order.push_back(v);
}
// 2. DFS на G^T в обратном порядке
void dfs2(int v, int c){
    comp[v]=c;
    for(int to: rg[v]) if(comp[to]==-1) dfs2(to,c);
}
for(int v=0;v<n;v++) if(!used[v]) dfs1(v);
fill(comp.begin(),comp.end(),-1);
for(int i=n-1;i>=0;i--)
    if(comp[order[i]]==-1) dfs2(order[i], cnt++);
```

Время: $O(V + E)$.

Поиск и восстановление циклов

Циклы в орграфе ищут через цвета вершин в DFS. Ребро в серую вершину (color==1) — обратное, цикл найден. Восстанавливают цикл по массиву предков p[], проходя от cyc_end до cyc_start. В неориентированном графе ребро к непосредственному родителю не считается циклом.

```
bool dfs(int v){
    color[v]=1;
    for(int to: g[v]){
        if(!color[to]){ p[to]=v; if(dfs(to)) return true; }
        else if(color[to]==1){ cyc_end=v; cyc_start=to; return true; }
    }
    color[v]=2; return false;
}
// Восстановление цикла:
vector<int> cycle;
for(int v=cyc_end; v!=cyc_start; v=p[v])
    cycle.push_back(v);
cycle.push_back(cyc_start);
reverse(cycle.begin(), cycle.end());
```

Время: $O(V + E)$.

Гамильтонов цикл при достаточных условиях

Теорема Дирака: если $n \geq 3$ и $\deg(v) \geq \frac{n}{2}$ для всех v — гамильтонов цикл существует. **Теорема Оре:** если для каждой пары несмежных вершин u, v выполнено $\deg(u) + \deg(v) \geq n$ — тоже существует. Оба условия достаточные, но не необходимые.

```
// Проверка условия Дирака
bool dirac = true;
for(int v=0;v<n;v++)
    if(deg[v] < n/2){ dirac=false; break; }

// Проверка условия Оре
bool ore = true;
for(int u=0;u<n;u++)
    for(int v=u+1;v<n;v++)
        if(!adj[u][v] && deg[u]+deg[v]<n)
            { ore=false; break; }
```

Время проверки: $O(n^2)$.

```
if(dirac || ore)
    cout << "Hamiltonian cycle exists\n";
```

Эйлеров цикл

Эйлеров цикл проходит по каждому ребру ровно один раз. Условия в неориентированном графе: все степени чётны + граф связан. В ориентированном: $\text{in}(v) = \text{out}(v)$ для всех v + граф связан. Алгоритм Хийерхольцера работает за $O(E)$.

```
// Проверка (неориент.): все степени чётны
for(int v=0; v<n; v++)
    if(deg[v]%2!=0){ cout<<"No Euler cycle"; return; }

// Хийерхольцер (с cur_edge – указатель текущего ребра)
vector<int> cur_edge(n, 0);
stack<int> st; vector<int> ans;
st.push(start);
while(!st.empty()){
    int v = st.top();
    if(cur_edge[v] < (int)g[v].size()){
        st.push(g[v][cur_edge[v]++]);
    } else {
        ans.push_back(v);
        st.pop();
    }
}
reverse(ans.begin(), ans.end());
```

Время: $O(E)$.

Компоненты связности

В неориентированном графе

Каждый запуск DFS или BFS из ещё не посещённой вершины выделяет ровно одну компоненту связности. Так можно не только посчитать число компонент, но и пронумеровать вершины по компонентам или собрать вершины каждой компоненты отдельно.

```
for(int v=0; v<n; ++v)
    if(!used[v]){ ++cc; dfs(v); }
```

Время: $O(V + E)$.

В ориентированном графе (Алгоритм Косораю)

1. Первый DFS (по исходному графу) + кладем вершину после обработки
2. Разворачиваем граф
3. Второй DFS (по обратному графу)

Время: $O(V + E)$.

Алгоритм Краскала

Алгоритм строит минимальное остовное дерево жадно: рассматривает рёбра в порядке возрастания веса и добавляет ребро, если оно соединяет разные компоненты. Для быстрой проверки используют DSU, поэтому алгоритм особенно удобен, когда список рёбер уже дан явно.

```
sort(edges.begin(), edges.end());
for(auto [w,u,v]: edges)
    if(find(u)!=find(v)) {
        unite(u,v);
    }
```

Время: $O(E \log E)$.

```
mst+=w;
}
```

Алгоритм Прима

Прим тоже строит минимальное остовное дерево, но растит его из одной стартовой вершины, каждый раз добавляя минимальное ребро на границе между уже выбранными и ещё не выбранными вершинами. На разреженных графах обычно используют кучу, а на плотных может быть удобен вариант с матрицей.

```
pq.push({0,s});
while(!pq.empty()){
    auto [w,v] = pop_min();
    if(in[v]) continue;
    in[v]=true;
    mst+=w;
    for(auto [to,c]: g[v])
        if(!in[to])
            pq.push({c,to});
}
```

Время: $O(E \log V)$ с кучей

Алгоритм Беллмана—Форда

Беллман—Форд находит кратчайшие пути из одной вершины даже при наличии отрицательных рёбер. Он последовательно расслабляет все рёбра, а дополнительная итерация после $n - 1$ проходов позволяет определить наличие достижимого отрицательного цикла.

```
d.assign(n, INF); d[s]=0;
for(int i=0;i<n-1;i++)
    for(auto [u,v,w]: edges)
        d[v]=min(d[v], d[u]+w);
```

Время: $O(V * E)$.

Алгоритм DAG (кратчайшие пути в DAG)

В DAG кратчайшие пути считаются быстрее, чем в общем случае, потому что после топологической сортировки рёбра можно релаксировать один раз в правильном порядке. Это даёт линейное по размеру графа решение и для положительных, и для отрицательных весов, если цикл отсутствует.

```
topo_sort(); d[s]=0;
for(int v: topo)
    for(auto [to,w]: g[v])
        d[to]=min(d[to], d[v]+w);
```

Время: $O(V + E)$.

Дейкстра (очередь / массив)

Дейкстра ищет кратчайшие пути из одной вершины в графе с неотрицательными весами, каждый раз фиксируя вершину с минимальной текущей дистанцией. Реализация с приоритетной очередью подходит для разреженных графов, а вариант с простым выбором минимума иногда используют на плотных.

```
d.assign(n,INF); d[s]=0;
priority_queue<P, vector<P>, greater<P>> pq;
pq.push({0,s});
while(!pq.empty()){
    auto [dist,v]=pq.top(); pq.pop();
    if(dist!=d[v]) continue;
```

Время: $O((V + E) \log V)$ с priority_queue, $O(V^2)$ с массивом.

```
for(auto [to,w]: g[v]) if(d[to]>d[v]+w){
    d[to]=d[v]+w; p[to]=v; pq.push({d[to],to});
}
}
```

Фloyd—Уоршалл

Фloyd—Уоршалл решает задачу кратчайших путей для всех пар вершин с помощью динамики по множеству разрешённых промежуточных вершин. Алгоритм прост в реализации и удобен, когда граф небольшой и нужно знать расстояния между всеми парами сразу.

```
for(int k=0;k<n;k++)
for(int i=0;i<n;i++)
for(int j=0;j<n;j++)
    d[i][j]=min(d[i][j], d[i][k]+d[k][j]);
```

Время: $O(V^3)$.

Диаметр дерева

Диаметр дерева можно найти двумя обходами: сначала из любой вершины ищут самую далёкую вершину A, а затем из A ищут самую далёкую вершину B. Путь между A и B и есть один из диаметров дерева, а его длина равна максимальному расстоянию.

```
int A = farthest(0);
auto [B, dist] = farthest(A);
```

Время: $O(V)$.

Кун: максимальное паросочетание в двудольном

Алгоритм Куна строит максимальное паросочетание в двудольном графе, пытаясь для каждой левой вершины найти увеличивающий путь. Если правая вершина уже занята, алгоритм пытается переназначить её текущую пару, чтобы освободить место для нового ребра.

```
bool try_kuhn(int v){
    if(used[v])
        return false;
    used[v]=true;
    for(int to: g[v]) {
        if(mt[to]==-1 || try_kuhn(mt[to])) {
            mt[to]=v;
            return true;
        }
    }
    return false;
}
```

Время: $O(V * E)$ в худшем случае.

Форд—Фалкерсон и Эдмондс—Карп (макс. поток)

Форд—Фалкерсон увеличивает поток, пока в остаточной сети есть путь *stot* с положительной пропускной способностью. Эдмондс—Карп выбирает **кратчайший** путь через BFS, что даёт полиномиальную оценку. Остаточная ёмкость: $r(u,v) = \text{cap}(u,v) - f(u,v)$.

```
// BFS augmenting path (Edmonds-Karp)
bool bfs(int s, int t, vector<int>& par){
    fill(par.begin(),par.end(),-1); par[s]=s;
    queue<int> q; q.push(s);
    while(!q.empty() && par[t]==-1){
        int v=q.front(); q.pop();
        for(int to: g[v])
```

Время: $O(VE^2)$
(Edmonds—Karp).

```

        if(par[to]==-1 && cap[v][to]-f[v][to]>0){
            par[to]=v; q.push(to);
        }
    }
    return par[t]!=-1;
}
int maxflow=0;
while(bfs(s,t,par)){
    int fl=INF;
    for(int v=t;v!=s;v=par[v])
        fl=min(fl, cap[par[v]][v]-f[par[v]][v]);
    for(int v=t;v!=s;v=par[v])
        { f[par[v]][v]+=fl; f[v][par[v]]-=fl; }
    maxflow+=fl;
}

```

Форда—Фалкерсон для max matching в двудольном

Максимальное паросочетание в двудольном графе можно свести к задаче максимального потока: добавляют исток, сток и рёбра единичной пропускной способности. После этого любой алгоритм maxflow автоматически находит размер паросочетания как величину максимального потока.

```

build_network(); // unit capacities
int matching = maxflow(s,t);

```

Время: как у выбранной реализации maxflow.

Алгоритм масштабирования потока

Масштабирование потока ускоряет поиск максимального потока при целых пропускных способностях: сначала ищут увеличивающие пути только по рёбрам остаточной сети с большой пропускной способностью, а затем постепенно уменьшают порог delta. Это уменьшает число «мелких» увеличений потока и часто работает быстрее обычного Форда—Фалкерсона.

```

int flow = 0;
int delta = highest_power_of_two_leq(U);
while(delta > 0){
    while(bfs_with_limit(s, t, delta, parent)){
        int add = bottleneck(parent);
        augment(parent, add);
        flow += add;
    }
    delta >>= 1;
}

```

Время: $O(E^2 \log U)$ для целых пропускных способностей, где U — максимальная capacity.

Минимальное вершинное покрытие в двудольном графе

В общем графе задача минимального вершинного покрытия NP-трудна, но в двудольном графе она решается через максимальное паросочетание по теореме Кёнига. После нахождения максимального паросочетания запускают обход по чередующимся путям из непокрытых вершин левой доли, а затем берут покрытие $(L \setminus Z_L) \cup (R \cap Z_R)$.

```

run_kuhn();
dfs_alt_from_unmatched_left();
cover = (L - visL) union (R & visR);

```

Построение покрытия после matching: $O(V + E)$. Итого: как поиск max matching, например $O(V * E)$ с Куном.

Максимальное независимое множество вершин в двудольном графе

В двудольном графе максимальное независимое множество является дополнением минимального вершинного покрытия. Поэтому достаточно найти минимальное покрытие, а затем взять все вершины, которые в него не вошли; по размеру получается $|MIS| = |V| - |MVC| = |V| - |MM|$.

```
cover = min_vertex_cover_bipartite();
for(int v = 0; v < n; ++v)
    if(!in_cover[v])
        indep.push_back(v);
```

Время: дополнение строится за $O(V)$. Итого: как поиск min vertex cover / max matching.

Мосты и точки сочленения

Мосты и точки сочленения ищут через один DFS, поддерживая время входа tin и значение low, которое показывает, насколько высоко по дереву можно подняться обратными рёбрами. Если из поддерева нельзя вернуться выше текущей вершины, соответствующее ребро или вершина оказываются критическими для связности.

```
void dfs(v,p=-1){
    used[v]=1;
    tin[v]=low[v]=timer++;
    for(int to: g[v]) {
        if(to != p) {
            if(used[to]) {
                low[v]=min(low[v], tin[to]);
            }
            else {
                dfs(to,v);
                low[v]=min(low[v], low[to]);
                if(low[to] > tin[v])
                    bridge(v,to);
            }
        }
    }
}
```

Время: $O(V + E)$.

Жадная раскраска графа

Жадная раскраска проходит вершины в некотором порядке и каждой вершине назначает минимальный допустимый цвет, который не конфликтует с уже раскрашенными соседями.

```
vector<int> color(n, -1);
for (int v = 0; v < n; ++v) {
    vector<bool> used(n, false);
    for (int to : g[v]) {
        if (color[to] != -1) {
            used[color[to]] = true;
        }
    }
    for (int c = 0; c < n; ++c) {
        if (!used[c]) {
            color[v] = c;
            break;
        }
    }
}
```

Время: $O(V + E)$

Полиномиальный хэш. Алгоритм Рабина—Карпа

Полиномиальный хэш строки: $h = s_0 \cdot p^0 + s_1 \cdot p^1 + \dots + s_{n-1} \cdot p^{n-1} \pmod{M}$. Рабин—Карп сдвигает окно за $O(1)$ с помощью **rolling hash**: вычитает первый символ, делит на p . При совпадении хэшей проверяет строки посимвольно.

```
const long long BASE=31, MOD=1e9+7;
// Построение префиксных хэшей
vector<long long> h(n+1,0), pw(n+1,1);
for(int i=0;i<n;i++){
    h[i+1]=(h[i]+(s[i]-'a'+1)*pw[i])%MOD;
    pw[i+1]=pw[i]*BASE%MOD;
}
// Хэш подстроки [l, r)
auto get=[&](int l,int r){
    return (h[r]-h[l]+MOD)%MOD;
};
// Рабин—Карп: поиск паттерна p в тексте t
long long hp=get_hash(p);
for(int i=0;i+m<=n;i++)
    if(get(i,i+m)==hp && t.substr(i,m)==p)
        cout<<i<<"\n";
```

Время: $O(n + m)$ в среднем.

Префикс-функция

π_i — длина наибольшего собственного префикса $s[0..i]$, который одновременно является суффиксом. Используется в КМП и для поиска всех вхождений паттерна.

```
vector<int> prefix(const string& s){
    int n=s.size();
    vector<int> pi(n,0);
    for(int i=1;i<n;i++){
        int j=pi[i-1];
        while(j>0 && s[i]!=s[j]) j=pi[j-1];
        if(s[i]==s[j]) j++;
        pi[i]=j;
    }
    return pi;
}
```

Время: $O(n)$.

Z-функция

$z[i]$ — длина наибольшего префикса строки s , совпадающего с подстрокой, начинающейся в позиции i . Поддерживается Z-box $[l, r)$: если $i < r$, то $z[i] \geq \min(r - i, z[i - l])$.

```
vector<int> z_function(const string& s){
    int n=s.size(); vector<int> z(n,0);
    z[0]=n; int l=0,r=0;
    for(int i=1;i<n;i++){
        if(i<r) z[i]=min(r-i, z[i-l]);
        while(i+z[i]<n && s[z[i]]==s[i+z[i]]) z[i]++;
        if(i+z[i]>r){ l=i; r=i+z[i]; }
    }
    return z;
}
```

Время: $O(n)$.

Алгоритм Кнута—Морриса—Пратта (КМП)

Склейка $p + \# + t$ позволяет найти все вхождения паттерна p в текст t за один проход: если $\pi_i = |p|$, то найдено вхождение в позиции $i - 2|p|$.

```
void kmp(const string& t, const string& p){
    string s = p + '#' + t;
    auto pi = prefix(s);
    int m = p.size();
    for(int i=m+1; i<(int)s.size(); i++){
        if(pi[i]==m)
            cout << i-2*m << "\n"; // позиция в t
    }
}
```

Время: $O(n + m)$.

Построение автомата для поиска подстроки

Конечный автомат строится на основе префикс-функции: таблица переходов $\delta[q][c]$ — из состояния q по символу c — вычисляется за $O(m \cdot |\Sigma|)$. После построения поиск в тексте длиной n занимает $O(n)$, состояние 0 — начальное, состояние m — паттерн найден.

```
// Построение автомата (используем префикс-функцию)
int m=p.size(); const int A=26;
vector<array<int,A>> aut(m+1);
aut[0].fill(0);
aut[0][p[0]-'a']=1;
for(int q=1;q<=m;q++){
    for(int c=0;c<A;c++){
        if(q<m && p[q]-'a'==c) aut[q][c]=q+1;
        else aut[q][c]=aut[pi[q-1]][c]; // pi — префикс-ф. p
    }
}
// Поиск: state=aut[state][t[i]-'a']
int state=0;
for(char ch: t){
    state=aut[state][ch-'a'];
    if(state==m) cout<<"/pos*/"match\n";
}
```

Построение: $O(m \cdot |\Sigma|)$.
Поиск: $O(n)$.

Бор (Trie, префиксное дерево)

Бор хранит множество строк в виде дерева, где каждое ребро помечено символом. Поиск и вставка — $O(|w|)$. Удобен для задач на префиксы и реализации Ахо-Корасика.

```
struct Trie {
    array<int,26> to;
    int cnt = 0; // слов оканчивается здесь
    Trie(){ to.fill(-1); }
};
vector<Trie> t(1);

void insert(const string& s){
    int v=0;
    for(char c : s){
        int k=c-'a';
        if(t[v].to[k]==-1){
            t[v].to[k]=t.size();
            t.emplace_back();
        }
        v=t[v].to[k];
    }
    t[v].cnt++;
}
```

Время: $O(|w|)$ на операцию.

```

bool find(const string& s){
    int v=0;
    for(char c : s){
        int k=c-'a';
        if(t[v].to[k]==-1) return false;
        v=t[v].to[k];
    }
    return t[v].cnt>0;
}

```

Алгоритм Ахо—Корасика

Строит автомат на Боре для поиска **множества** паттернов одновременно. Fail-ссылки (суффиксные ссылки) — аналог КМП-префикс-функции на Боре: из вершины v с меткой $u[0..k]$ ссылка ведёт к вершине с меткой максимального суффикса $u[0..k]$, являющегося префиксом какого-либо паттерна.

```

struct AhoCorasick {
    vector<array<int,26>> go;
    vector<int> fail, out; // out — битмаска паттернов
    AhoCorasick(): go(1), fail(1,0), out(1,0){}

    void add(const string& s, int id){
        int v=0;
        for(char c:s){
            int k=c-'a';
            if(!go[v][k]){
                go.push_back({}); fail.push_back(0);
                out.push_back(0);
                go[v][k]=go.size()-1;
            }
            v=go[v][k];
        }
        out[v]|=(1<<id);
    }

    void build(){
        queue<int> q;
        for(int c=0;c<26;c++) if(go[0][c]) q.push(go[0][c]);
        while(!q.empty()){
            int v=q.front(); q.pop();
            out[v]=out[fail[v]]; // суффиксные вхождения
            for(int c=0;c<26;c++){
                if(go[v][c]){
                    fail[go[v][c]]=go[fail[v]][c];
                    q.push(go[v][c]);
                } else go[v][c]=go[fail[v]][c];
            }
        }
    }
};
// Поиск: state=ac.go[state][c-'a']

```

Время: $O(\sum |p_i| \cdot |\Sigma| + n + z)$.

Хэширование: идея, хэш-таблица, коллизии

Хэш-функция $h : U \rightarrow \{0, \dots, m-1\}$ отображает ключи в индексы таблицы. Идеальная функция — инъекция, реальная — допускает коллизии. Простой пример: $h(k) = k \bmod m$, m — простое. **Коллизия** — $h(k_1) = h(k_2)$ при $k_1 \neq k_2$.

Поиск/вставка: $O(1)$ в среднем, $O(n)$ в худшем.

Открытая и закрытая адресация

Метод цепочек (закрытая): каждый слот — связный список. **Открытая адресация:** при коллизии ищем следующий свободный слот по правилу пробирования.

```
// Метод цепочек
vector<list<int>> table(m);
table[h(key)].push_back(key);

// Линейное пробирование (открытая адресация)
int i=h(key);
while(table[i]!=EMPTY) i=(i+1)%m;
table[i]=key;

// Квадратичное: i = (h(k) + j^2) % m
// Двойное хэширование: i = (h1(k) + j*h2(k)) % m
```

Время: $O\left(\frac{1}{1-\alpha}\right)$, где $\alpha = \frac{n}{m}$ — коэффициент загрузки.

Метод Кукушки

Используются две хэш-таблицы T_1 и T_2 с функциями h_1, h_2 . Вставка: поместить в $T_1[h_1(k)]$; если занято — вытолкнуть старый ключ в $T_2[h_2(\cdot)]$, и так далее. При цикле ($> N$ перемещений) — перехэширование. Гарантированный $O(1)$ поиск.

```
void insert(int key){
    for(int i=0; i<MAX_IT; i++){
        if(T1[h1(key)]==EMPTY){ T1[h1(key)]=key; return; }
        swap(key, T1[h1(key)]);
        if(T2[h2(key)]==EMPTY){ T2[h2(key)]=key; return; }
        swap(key, T2[h2(key)]);
    }
    rehash(); insert(key); // цикл → перехэширование
}

bool find(int key){
    return T1[h1(key)]==key || T2[h2(key)]==key;
}
```

Время: $O(1)$ поиск/удаление; вставка $O(1)$ амортизировано.

Фильтр Блума

Вероятностная структура данных для проверки принадлежности к множеству. Битовый массив размером m , k хэш-функций. **Ложноположительный** ответ возможен, **ложноотрицательный** — нет. Вероятность ложного срабатывания: $\approx (1 - e^{-kn/m})^k$.

```
bitset<M> bloom;

void add(const string& s){
    for(int i=0; i<k; i++){
        bloom.set(hash_i(s, i)%M);
    }
}

bool query(const string& s){
    for(int i=0; i<k; i++){
        if(!bloom.test(hash_i(s, i)%M))
            return false; // точно нет
        return true; // возможно есть
    }
}

// Удаление невозможно (только counting Bloom filter)
```

Память: $O(m)$ бит;
вставка/запрос: $O(k)$.